The Relay: write-up

1 What I built and why

A Python/asyncio pipeline over the three pinned PDFs (pdftotext; chunks never cross a page, so a citation is a page). Steps: **understand** (small model rewrites the query, hints which paper) → **retrieve** (own BM25, optional lexical re-rank, paper routing) → **tool** (literal search for numbers, acronyms and quoted strings; a branch, skipped silently if slow or failing) → **draft** (big model, streamed) → **verify**, deterministic and sentence by sentence: each sentence needs a valid [paper-page-chunk] citation, at least 50% of its content words and all its numbers must appear in the cited passage. A failed check before anything is shown triggers a **repair loop** (one more draft, 3 passages, 'quote exactly'), then a **no-LLM excerpt fallback**. Around it: a per-model governor (RPM/TPM buckets at 92% of the limit, in-flight cap, deadline-ordered queue that sheds a request it cannot serve in time, believed limits cut by 30% on each 429), a cache with single-flight for identical questions, and a retry budget. **The decision that shaped it most:** degrade to a cheaper answer that is still checked, never to an unchecked one. Nothing reaches the person unless it passed verification, so 'first token' means the first *verified* sentence. When the governor predicts the big model cannot answer inside my assumed patience (first text by 8 s, finished by 25 s), the person gets a cited excerpt or an honest refusal rather than a queue. Cost: that excerpt path answers only 11 of my 24 questions correctly (offline test), and it wrongly refuses more than the full path (section 5).

2 The real service and my stand-in

Real service. Groq's Llama model IDs I first assumed returned 404; `/models` listed `openai/gpt-oss-20b` and `-120b` (reasoning models; I set reasoning_effort=low). Limits (Table 1) came from: `x-ratelimit-*` headers on 45 paced calls; 30 back-to-back tiny calls (30 succeeded, the 31st got 429 with retry-after 2 s, and calls worked again after 2.4 s); concurrency 1 to 16 (never refused); an oversized prompt (HTTP 413, not 429: 'Limit 8000, Requested 9079'); and by really exhausting the 120B model: 429 'tokens per day (TPD): Limit 200000, Used 199107 ... try again in 31.5s'. Reserved output counts: remaining-tokens dropped by 1,738 at call start = 1,538 prompt + 200 max_tokens. **Stand-in** (`standin/server.py`): OpenAI-compatible SSE; token buckets for RPM and TPM, hard RPD/TPD, Groq-format 429s with headers and retry-after; lognormal time to first token fitted from the 45 real samples (20B: median 543 ms, sigma 0.24; 120B: 566 ms, 0.31; tokens/s from the usage field); service time grows by 1+1.2(in-flight−6)/6 beyond 6 in flight; 3% of calls 'succeed slowly' (8x), 1% fast 5xx, 0.5% stall mid-stream; replies stream in 16-character pieces. **Least like the real thing:** (i) its answers are extracted from the prompt, so stand-in grades test the plumbing, not model quality; (ii) the slow tail, the load knee, the 5xx rate and 'a rejected request still counts against RPM' are assumptions: I saw no slow tail in 45 real calls and no slowdown up to 16 in flight; (iii) it resets daily limits hard, while the real TPD is a rolling window; (iv) no 413s, no reasoning-token delay, no format drift (real 120B wrote its citations in fullwidth brackets with narrow spaces; the verifier normalises them).

3 Results

Tables 1 to 3 are on the pages after the write-up; `make_tables.py` recomputes them from `results.csv` (478 rows; rows prefixed 'contaminated_' are real runs spoiled when the 120B model ran out of daily tokens: kept in the file, left out of the tables). Real runs: Table 3, real_steady and real_burst (20B for both calls, since 120B was out of tokens), real_daily_exhausted (120B really out). All other rows are stand-in. Percentiles are linear-interpolated over requests that received text; **n is 5 to 63 per row, so p95/p99 are near the maximum, not stable estimates. A number I could not have guessed:** tokens per minute, not requests per minute, is the binding limit. 8,000 TPM / ~780 tokens per answer (120-word pieces) is about 10 answers a minute; 30 RPM / 2 calls is 15. With 250-word pieces (~1,500 tokens) it is 5 a minute, with 500-word pieces (~2,650) it is 3. So piece size is a latency setting: median first token was 2.6 s (120-word) vs 10.5 s (250) vs 19.4 s (500) even though model time to first token was 0.5 to 1.1 s in all three; the rest is queueing. In the burst, 23 of 63 requests were answered from cache or single-flight for zero tokens (tokens per answer 116 over all, 814 for those that used the model). **Quality** (24 questions I wrote, 18 answerable and 6 the papers cannot answer; real gpt-oss-20b, no load; Table 3): the chosen setting got 62.5% correct and cited, 8.3% wrong, 25.0% correctly refused (6 of 6) and 4.2% wrongly refused, at 778 tokens per answer. **Caveats:** I changed the grader once after reading results (v1 demanded a cited page from my gold list and marked 3 correct answers wrong because they cited another page that states the same fact), then re-ran everything. The refusal gate (0.40), verifier (0.5) and excerpt bar (0.60) were tuned on these same 24 questions, so quality is optimistic.

4 Where the time goes

Slowest real request: `retr-s500_k3_rerank-0-q09`, 45,188 ms total = **queue 43,572 ms + working 1,615 ms + retrying 0 ms** (+2 ms untracked). All of it came **before** the first token (shown at 45,188 ms, i.e. after the last repair call; model time to first token 0.8 s). How I know: every call is timed in the client (`queue_ms` = awaiting the governor, `work_ms` = successful calls plus retrieval/verify, `retry_ms` = failed attempts plus backoff) and the three sum to total within 2 ms; the governor's per-step trace shows the request waiting for TPM tokens (each 500-word answer reserves ~2.6k of 8k per minute) and then a second draft after a failed verification. It was request 9 of a back-to-back run with 180 s deadlines, so it is the cost of the setting at the limit, not a provider slowdown.

5 When a limit is hit

Per minute: the governor paces to 92% of RPM/TPM, serves waiters by deadline, and sheds a head it cannot serve in time to the excerpt path; at arrival a request is told 'excerpt' if its estimated wait exceeds 70% of its remaining first-token time. A 429 pauses the lane for retry-after and cuts the believed limit by 30%. Stand-in burst (60 requests in 4 s plus 3 a minute, 63 total): 0 rate-limited, 0 given up; 23 served free from cache/coalescing, 20 full answers, 19 excerpts, 1 partial. **Delayed:** later arrivals (queued). **Dropped:** nobody. **Weaker:** whoever arrives after the budget is committed; on stand-in content 17 of 63 were wrongly refused and 12 wrong, so treat the degraded path as the real cost of 'nobody turned away'. **Per day:** a TPD/RPD 429, or my local tally reaching zero, sends every request to excerpts, and the lane stays blocked at least 5 minutes (TPD is rolling: 'try again in 31 s' frees about 70 tokens, not an answer). Real run, 120B out of tokens: 11 requests, 0 failed, 0 given up, 6 got excerpts (p50 total 52 ms), 4 of 11 wrongly refused. Why: queueing for a budget that returns at ~2.3 tokens/s would hold people for minutes; an excerpt costs no tokens, and the strict excerpt bar prefers refusing to quoting the wrong passage.

Retrieval: quality versus cost

Four settings, all 24 questions, real 20B, one at a time, no load (Table 3). Evidence in the retrieved set for answerable questions: 16, 17, 16, 15 of 18. 120 words/k=3: 62.5% correct, 778 tokens, first token p50 2.6 s. 250/k=4: same 62.5% correct at 1,517 tokens and 10.5 s. 250/k=4 with re-rank: 62.5% correct, more wrong (12.5%). 500/k=3 with re-rank: worse (58.3%, 16.7% wrong, 2,655 tokens, 19.4 s). **Chosen: 120 words, 3 pieces, no re-rank**: equal quality at half the tokens and a quarter of the wait; my lexical re-rank bought nothing. Caveat: 24 questions cannot separate 62.5% from 58.3%. **Near the limit:** skip the understand call, keep k=3, check the cache first, and past the deadline estimate fall back to excerpts over the same 3 pieces.

How this library will mislead a naive retriever

(1) **Symbols versus words.** The PDF says β2 = 0.98 and ε = 10–9; users write 'beta2'. Q: 'Which optimizer was used to train the Transformer, and what were its beta2 and epsilon?' scored 0.27 coverage, my refusal gate rejected it and the answer was a wrong refusal until I mapped Greek letters to names. (2) **One vocabulary, three papers.** 'GPU', 'tokens', 'Transformer' appear everywhere and Moshi is 51% of all chunks. Q: 'How many H100 GPUs were used to train the H-Net models?' (unanswerable) returns Moshi's 'All models are trained on H100 GPUs' first; a naive system answers with Moshi's setup. (3) **Repetition crowds out the one page that answers.** Mimi's 12.5 Hz appears on many Moshi pages; Q: 'What frame rate and bitrate does the Mimi codec operate at?' retrieved pages 23, 12 and 53 but not page 11, which holds '1.1 kbps', so the model correctly said 'not in the papers' and was wrong.

6 Retries

Retry only 429, 5xx, stalls and timeouts; never 400/413/401. Sleep retry-after plus up to 20% jitter, else full-jitter exponential (0.5 s x 2^n, cap 8 s); at most 2 retries; only if backoff plus the recent service time fits the remaining deadline; a global budget of 20% of first attempts (3 to start); a daily-limit 429 is not retried for 5 minutes. **Where retrying made things worse:** stand-in sustained overload (27 requests a minute) with a naive client (5 immediate retries, no governor, no budget) sent 89 provider calls for 54 requests versus 53 for the governed client, collected 12 429s, and lost 2 requests (96.3% succeeded versus 100%), with p99 first token no worse than the governed run (5.4 s vs 5.9 s), so the cost was load and failures, not latency. If a rejected request also consumes RPM, which I assumed in the stand-in and did not test on Groq, each retry steals capacity from the next person. Honest limit: in this run the damage was extra load and 2 failures, not a collapse. On the real service honouring retry-after worked (recovered in 2.4 s after a 2 s hint), but the TPD hint ('31 s') would have been a trap: waking up then frees ~70 tokens, so retrying then is wasted traffic.

7 What the person sees

At 3 pm, daily budget gone, a waiting person gets an answer in about 50 ms, not a spinner: either 'Today's model budget is used up, so I can't write a full answer right now.' followed by the best one or two sentences from the papers in quotation marks, each with its [paper-page-chunk] citation, or 'The three papers (Attention Is All You Need v7, Moshi v1, Dynamic Chunking v2) don't say. I'd rather tell you that than guess.' Under minute-level load the lead-in is 'I'm under heavy load, so instead of a written answer here is the closest text from the papers.' **What is wrong with it:** it does not say when full answers return (about one answer every 6 minutes from the rolling refill), and in degraded mode 'the papers don't say' is not true: it means 'I did not find a close enough passage'. I measured that cost (4 of 11 wrongly refused) but did not fix the wording.

8 What I'm unsure about, what I left out, what I learned

Least certain assumption: patience (first text by 8 s, finished by 25 s). I did not measure it. If people wait 20 s, I turn many requests into excerpts for nothing (19 of 63 in the burst); if they leave at 3 s, even full answers (p50 2.6 s, p95 about 9 s at the limit) arrive too late. Second: the 24 questions tune and test the same thresholds. **Deliberately not built:** (a) duplicate ('hedged') calls for slow responses: I saw no slow tail in 45 real calls to justify the extra token spend; (b) embedding or cross-encoder retrieval, and 'excerpt now, upgraded answer later'. **New to me, in my words:** a provider's per-day token limit can be a sliding window, not a midnight reset. After 'Used 199,107 of 200,000' the error said 'try again in 31 s', which is just the time for 73 tokens to drain back at 200,000/86,400 = 2.3 tokens a second. 'Out of budget' is therefore a slope: one ~800-token answer comes back every ~6 minutes. And the per-minute token limit is charged for max_tokens at call start, not for what the model writes, so a generous max_tokens lowers throughput.

Your feedback on Vaani

(Draft from reading the site text, not from using the product: replace with your own view.) What stood out: the focus on Indian languages and mid-call switching (a Hindi reschedule example), concrete customer numbers (891 leads qualified, 16,282 calls), and the Build, Run, Improve loop with 'Teacher AI' that builds an agent from one sentence. What confused me: headline claims with no definition: 'the only voice AI harness you'll ever need', 'p90 ~450 ms' and 'response accuracy on noisy telephony audio'. Measured

from where to where, in which language, on what traffic, and accurate at what? I would change one thing: replace the single p90 with a live latency distribution (p50/p90/p99 by language and model stack) and define accuracy: the lesson of this assignment is that one number hides the tail.

How I worked with the assistant

Full uncleaned transcript: `conversation_history/` (API key redacted). **Things the assistant got wrong, and how I caught them (from the transcript):** it assumed the Llama model IDs (404 on the first real probe); its planner only counted queued work, so 14 simultaneous real requests all chose the full path and 10 gave up (caught in real_burst: 71% given up); its grader rejected right answers (caught by reading the 5 failing cases one by one); its local daily-limit tally locked the system into excerpts after the real TPD was reached (caught when 'extractive:daily' appeared while the API still showed 941 requests left). **The prompts that mattered (my real messages, quoted from the transcript):** (1) 'complete this task/project/assignement to the best of the knowledge and make it surpass': set the scope and the bar, and everything else was the assistant's plan. (2) 'what things need to be put in': I asked what the submission must contain before trusting it was complete. (3) 'use this key': I authorised real Groq calls, which is why the limits, the 413, the concurrency test and the daily-token window are measured, not assumed. **Honest account of my role:** my prompts were short and high level; the design choices (governor, thresholds, grader, stand-in parameters) were proposed by the assistant and I have read them to be able to defend them. **[YOU: add, only you know it]** One decision where you overruled the assistant (the transcript contains none, so write one only if it really happened), and what in this submission is distinctly yours.

Required tables (generated by make_tables.py from results.csv; first token = first verified text shown)

Table 1: Limits found in the real service (gpt-oss-20b / gpt-oss-120b, free tier, 2026-11-11)

Limit	Value found	How found	Confidence
Requests per minute	30 (both models)	30 tiny back-to-back calls OK, 31st got 429 'RPM: Limit 30, Used 30' (20B probed; 120B same plan, not separately probed)	high (20B), medium (120B)
Tokens per minute	8,000 (both)	x-ratelimit-limit-tokens header; 413 'Limit 8000, Requested 9079'; remaining dropped by prompt+max_tokens at call start	high
Requests per day	1,000 (both)	x-ratelimit-limit-requests header; remaining count fell as expected (never exhausted)	high (value), not tested by exhaustion
Tokens per day	200,000 (120B; 20B not hit)	429 'TPD: Limit 200000, Used 199107, Requested 966' after the evaluation runs	high (120B), low (20B)
Requests in flight at once	no cap found up to 16	1, 2, 4, 8, 16 simultaneous tiny calls all returned 200	medium (tiny prompts only)
What it returns when a limit is hit	RPM/TPD/TPM: HTTP 429 JSON with retry-after (s) and message naming the limit; oversize request: HTTP 413	raw/probe_real.jsonl	high
Time until it recovers	RPM: 2.4 s (hint 2 s). TPD: rolling, 'try again in 31.5 s' for a 73-token shortfall (about 2.3 tokens/s)	recovery poll; TPD error text	medium

Table 2a: Latency (ms; 'stand-in' rows are stand-in; real rows in the table)

Scenario	Real or stand-in	Requests sent	Duration (s)	Offered rate (per min)	First token p50 / p95 / p99 (ms)	Total time p50 / p95 / p99 (ms)
steady	stand-in	8	108.4	4.0	1204 / 4827 / 4974 (n=8)	1249 / 4875 / 5016 (n=8)
burst	stand-in	63	77.5	42.0	2 / 1547 / 2651 (n=63)	51 / 1624 / 2734 (n=63)
overload	stand-in	54	119.8	27.0	633 / 2586 / 5856 (n=54)	662 / 2630 / 5995 (n=54)
rpm_halved	stand-in	11	119.2	5.5	1238 / 3615 / 5241 (n=11)	1280 / 3678 / 5313 (n=11)
daily_exhausted	stand-in	17	86.5	10.2	1075 / 1640 / 1967 (n=16)	1123 / 1721 / 2060 (n=16)
overload_naive_retry	stand-in	54	119.8	27.0	862 / 1793 / 5421 (n=52)	940 / 1838 / 5509 (n=52)
overload_no_cache	stand-in	54	120.2	27.0	665 / 2454 / 5865 (n=54)	690 / 2522 / 6005 (n=54)
real_steady	real	5	80.9	3.3	584 / 789 / 792 (n=5)	618 / 822 / 823 (n=5)
real_burst	real	14	3.5	18.7	9 / 1011 / 1120 (n=14)	52 / 1037 / 1143 (n=14)
real_daily_exhausted	real	11	58.9	11.0	16 / 1834 / 2104 (n=11)	52 / 1859 / 2122 (n=11)

Table 2b: Outcomes (cost = list price per 1,000 answers, from remembered prices: unverified; the free tier cost $0)

Scenario	Succeeded %	Rate-limited %	Timed out or given up %	Retries per request	Peak queue length	Tokens per answer (all / model-path only)	Cost per 1,000 answers (list-price USD)
steady	100.0	0.0	0.0	0.12	0	595 / 595	$0.11
burst	100.0	0.0	0.0	0.02	0	116 / 814	$0.02
overload	100.0	0.0	0.0	0.00	1	418 / 706	$0.07
rpm_halved	100.0	0.0	0.0	0.00	0	642 / 706	$0.11
daily_exhausted	94.1	0.0	5.9	0.06	0	585 / 624	$0.10
overload_naive_retry	96.3	3.7	0.0	0.33	0	489 / 620	$0.09
overload_no_cache	100.0	0.0	0.0	0.00	1	446 / 651	$0.08
real_steady	100.0	0.0	0.0	0.00	0	589 / 589	$0.12
real_burst	100.0	0.0	0.0	0.00	0	247 / 866	$0.05
real_daily_exhausted	100.0	0.0	0.0	0.00	1	176 / 388	$0.03

Table 3: Retrieval settings (real gpt-oss-20b, 24 questions, no load; first-token time includes governor queueing, see section 3)

Setting (piece words, pieces fetched, re-rank)	Source	Questions asked	Correct and cited %	Wrong %	Correctly refused %	Tokens per answer	First token p50 / p95 (ms)	Total time p50 / p95 (ms)
120 words, k=3, rerank=no	real	24	62.5	8.3	25.0	778	2562 / 8918	2562 / 8918
250 words, k=4, rerank=no	real	24	62.5	8.3	25.0	1517	10490 / 23743	10490 / 23756
250 words, k=4, rerank=yes	real	24	62.5	12.5	25.0	1448	10300 / 20805	10300 / 20805
500 words, k=3, rerank=yes	real	24	58.3	16.7	25.0	2655	19429 / 43865	19430 / 43866

Path/grade breakdown, a 'rate-limited' reading, and per-row detail are in tables.md and results.csv (columns path, mode, n429, queue_ms, work_ms, retry_ms, other_ms). 'Rate-limited %' = requests that ended with no answer because of limits; the share that saw any 429 is in tables.md.